

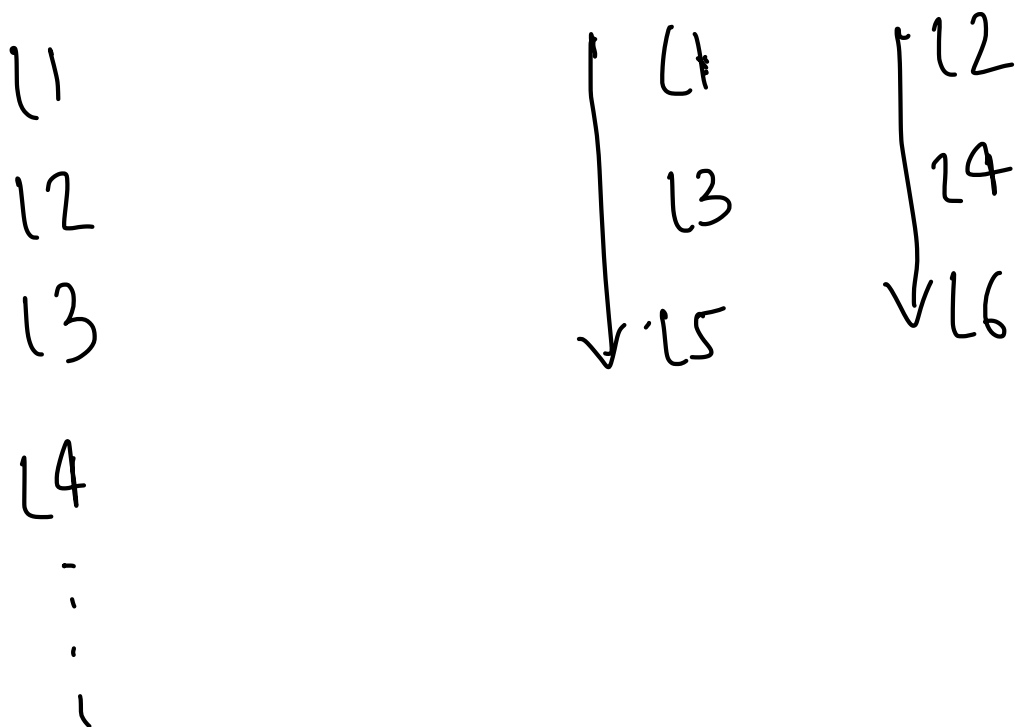
Threads

An executing/running program is process.

If $\Downarrow$ this runs sequentially, then the entire code has to be run in coordination.

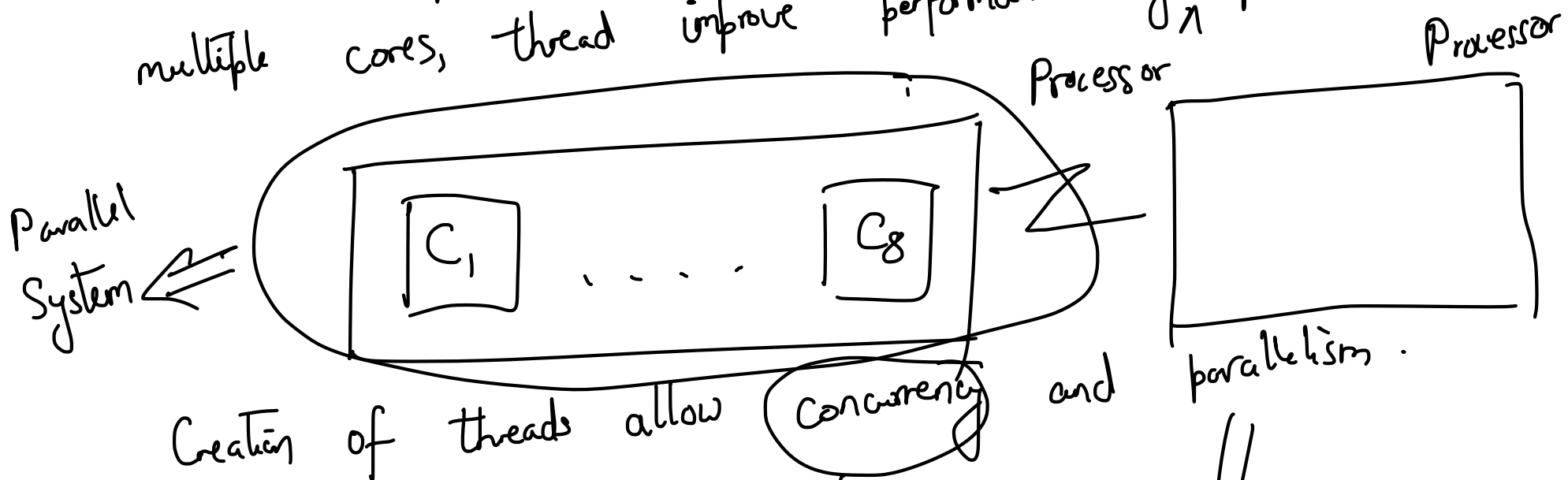
- ① No need to coordinate the tabs of a browser
- ② GUI and app logic $\Rightarrow$ Only occasionally the GUI needs to fetch data from the backend logic.

Threads $\Rightarrow$ independent line of execution of a program.



In Linux, all processes have a pid. If a thread is created, then only the pid is copied, but not any extra memory regions.

With all processors of devices we are used to having multiple cores, thread improve performance by enabling parallel execution.



Creation of threads allow Concurrency and

parallelism.
Some instructions are executed at the same time.

Do not care about the sequence of execution

```
#include <pthread.h>
int a = 5;
void subtask() { a = a - 1; }

int main() {
    pthread* tid = pthread_create(&subtask());
    tid->run();    a = a + 1;
}
```

```

    tid → join();
    return 0;
}

```

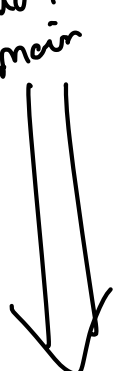
Suppose there is a global variable that we need to manipulate. 'a' is a global variable. What will be the output? This has non-deterministic output and depends on the sequence of execution. This is called race condition and should be avoided. Some sort of locking mechanism is needed to avoid such race conditions.

```
#include <thread>
```

```
int main()
```

```
{
    Thread t(<name of function>, <args of that
    function>);
    t.run();
    t.join();
    t.detach();
    t.yield();
    t.sleep();
}
```

Execution
part of main
thread



Common functions in the thread class to manipulate threads.

Documentation says some data structures are

thread-safe

Locks are already used

to ensure no race conditions happen due to creation of multiple threads

```
for (int i = 0; i < 100; i++) {  
    A[i] = B[i] + C[i];  
}
```

$A_v = B_v + C_v$ } Data-level parallelism

Python does not allow execution of parallel threads.

[Global Interpreter Lock]

But separate processes are run in parallel.

joblib library.

top/htop command to monitor the usage of cores.

No race conditions are possible in Python.

} 20% of the total time

for (int i = 0; i < 100; i++) {
 $A[i] = B[i] + C[i];$
}

} 80% of total time

8 cores; so

$$0.1x + 0.2x = 0.3x$$

The benefits of parallelism can be bottlenecked by the portions that must be executed sequentially.

Amdahl's Law